

AgriVista Field

DOSSIER TECHNIQUE

Développement mobile Flutter / Dart 3

Master BIHAR — Session 2025-2026

Auteur : Dorian ARGAILLOT

Version 1.0 — 4 septembre 2026

Table des matières

1. Présentation du projet.....	3
2. Stack technique et versions.....	3
3. Architecture du projet.....	4
4. Organisation des fichiers.....	6
5. Gestion d'état avec Riverpod.....	6
6. Chargement et désérialisation JSON.....	7
7. Persistance locale avec Hive.....	7
8. Gestion des erreurs.....	8
9. Tests et validation.....	9
10. Difficultés et solutions.....	10
11. Limites.....	11
12. Améliorations futures.....	11
13. Conclusion.....	12
Annexe A — Matrice de conformité au sujet.....	12
Annexe B — Livraison et versionnement.....	14

1. Présentation du projet

1.1 Contexte

AgriVista est une entreprise corse spécialisée dans l'installation et la maintenance de stations de capteurs connectés pour les exploitations agricoles et viticoles. Ses techniciens interviennent sur le terrain, parfois avec une couverture réseau faible ou intermittente.

AgriVista Field constitue une première version de l'application interne de gestion des interventions. Elle remplace la consultation papier par une liste mobile structurée, permet d'accéder aux informations nécessaires à une intervention et d'en faire évoluer le statut.

1.2 Objectifs couverts

Le périmètre réalisé correspond au socle obligatoire du sujet :

- lecture et désérialisation du JSON fourni ;
- liste, recherche et filtres par statut et priorité ;
- détail d'une intervention ;
- progression `planifiee` → `en_cours` → `terminee` ;
- persistance locale des statuts ;
- états `loading`, `data`, `empty` et `error` ;
- `retry` manuel ;
- thème Material 3 et profil simple ;
- structure Data / Domain / Presentation avec Riverpod.

Aucun backend ni aucune fonctionnalité optionnelle n'a été ajouté.

2. Stack technique et versions

Les versions ont été relevées dans l'environnement Flutter et dans `pubspec.lock`.

Composant	Version	Utilisation
Flutter	3.44.8	Framework de l'application Android/iOS
Dart	3.12.2	Langage, programmation asynchrone et Domain
Material 3	SDK Flutter 3.44.8	Design system, thème et composants UI
flutter_riverpod	3.4.2	Injection et gestion d'état réactive
dio	5.11.0	Requête GET vers le JSON distant
freezed	3.2.5	Génération des DTO immuables
freezed_annotation	3.1.0	Annotations Freezed

json_serializable	6.14.1	Génération de fromJson/toJson
json_annotation	4.12.0	Annotations de sérialisation
hive_flutter	1.1.0	Initialisation de Hive avec Flutter
hive	2.2.3	Stockage clé-valeur des statuts
build_runner	2.15.1	Orchestration de la génération de code
flutter_lints	6.0.0	Règles de qualité statique

Les packages tiers sont réutilisés depuis pub.dev (<https://pub.dev/>) et restent soumis à leurs licences open source respectives.

2.1 Environnement requis

- Flutter et Dart dans les versions indiquées ou compatibles ;
- Android SDK avec appareil ou émulateur pour Android ;
- macOS et Xcode pour construire et tester iOS ;
- connexion réseau pour le premier chargement du JSON.

La compilation iOS n'a pas pu être exécutée dans l'environnement Windows utilisé pour le développement.

3. Architecture du projet

3.1 Principe

L'architecture suit le découpage :

Presentation → Domain ← Data

Le Domain porte les abstractions et les règles. La Data implémente ces abstractions. La Presentation ne manipule que des entités et use cases exposés par Riverpod. L'assemblage concret est effectué au démarrage par le composition root.

```

UI
↓
Riverpod
↓
Use Case
↓
Repository abstrait
↓
Repository Data
↓
RemoteDataSource Dio + LocalDataSource Hive

```

3.2 Domain

Le Domain contient :

- les entités `Technicien`, `Intervention`, `ActionHistorique` et `DonneesInterventions` ;
- les énumérations `Priorite` et `StatutIntervention` ;
- le contrat `InterventionRepository` ;
- les use cases de chargement et de mise à jour ;
- la règle de calcul du prochain statut.

Il reste en Dart pur. L'absence d'import Flutter, Riverpod, Dio ou Hive permet de tester les règles métier sans framework et d'éviter le couplage aux détails techniques.

3.3 Data

La couche Data contient :

- `DioInterventionRemoteDataSource` pour la lecture distante ;
- `HiveInterventionLocalDataSource` pour les statuts locaux ;
- les DTO `Freezed/json_serializable` ;
- les mappers `DTO → Domain` et `stockage → statut` ;
- `InterventionRepositoryImpl` pour la fusion distant/local.

3.4 Presentation

La Presentation regroupe :

- les pages liste et détail ;
- la page de profil ;
- les widgets de cartes, filtres, badges et historique ;
- l'`AsyncNotifier` des interventions ;
- le `Notifier` des critères de recherche et de filtres ;
- les vues communes `loading`, `error` et `empty`.

3.5 Composition root

`bootstrap.dart` initialise Hive, construit Dio, les Data Sources et le repository, puis injecte ce dernier dans `ProviderScope`. Les implémentations techniques ne sont donc pas construites par les widgets.

4. Organisation des fichiers

```
lib/
├── app/
│   ├── app.dart
│   ├── app_constants.dart
│   ├── app_shell.dart
│   └── bootstrap.dart
├── core/
│   ├── errors/app_failure.dart
│   ├── network/dio_client.dart
│   ├── theme/app_theme.dart
│   ├── utils/date_formatter.dart
│   └── widgets/async_state_views.dart
├── features/
│   ├── interventions/
│   │   ├── data/
│   │   │   ├── datasources/
│   │   │   ├── mappers/
│   │   │   ├── models/
│   │   │   └── repositories/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   ├── repositories/
│   │   │   └── usecases/
│   │   └── presentation/
│   │       ├── pages/
│   │       ├── providers/
│   │       └── widgets/
│   ├── profile/
│   │   ├── presentation/
│   │   │   ├── pages/
│   │   │   └── widgets/
└── main.dart
```

`app/` contient l'entrée applicative, la navigation et l'injection. `core/` rassemble les éléments transverses. Chaque fonctionnalité conserve ses responsabilités propres dans `features/`.

5. Gestion d'état avec Riverpod

5.1 Providers de dépendances

Des `Provider` exposent le contrat du repository et construisent les use cases. L'implémentation Data est fournie par surcharge au niveau du `ProviderScope` racine. Les tests remplacent ces providers par des fakes isolés.

5.2 État asynchrone principal

`interventionsProvider` est un `AsyncNotifierProvider<InterventionsNotifier, DonneesInterventions>`. Il constitue la source de vérité commune pour :

- la liste ;
- le détail ;
- le profil et son technicien.

`AsyncValue` représente les états loading, data et error. Le retry automatique de Riverpod est désactivé afin de respecter le choix d'un rechargement exclusivement manuel.

5.3 Recherche et filtres

Un `Notifier` indépendant conserve le texte de recherche, le statut sélectionné et la priorité sélectionnée. La recherche reste locale et porte sur la station, le domaine et la description.

`ref.watch` observe les données ou les filtres. `ref.read` est utilisé pour les actions ponctuelles : modifier un filtre, recharger ou progresser dans les statuts.

6. Chargement et désérialisation JSON

6.1 Source

```
https://utrera.ludovic.aflokkat-projet.fr/getInterventions.json
```

La ressource est appelée par une requête GET et reste strictement en lecture seule. Aucun backend n'a été développé et aucune écriture distante n'est réalisée.

6.2 Chaîne de traitement

```
Dio
↓
DioInterventionRemoteDataSource
↓
AgriVistaResponseDto / InterventionDto
↓
Freezed + json_serializable
↓
Mappers
↓
DonneesInterventions / Intervention
```

La conversion traduit notamment :

```
JSON "en_cours" → StatutIntervention.enCours
date ISO "2026-06-15" → DateTime
```

Le mapping refuse les champs obligatoires vides, les statuts ou priorités inconnus, les dates invalides et les coordonnées non finies ou hors limites géographiques. Une structure JSON incorrecte est transformée en `DataParsingFailure`. Le chargement échoue dans son ensemble afin de ne jamais afficher silencieusement une liste partiellement corrompue.

7. Persistance locale avec Hive

7.1 Structure

La box utilisée est :

```
intervention_statuses
```

Son contenu est minimal :

```
interventionId → statut
```

Par exemple :

7.2 Fusion distant/local

```
interventions du JSON
+
surcharges de statut Hive
↓
liste finale du Domain
```

Le JSON reste la référence pour l'existence et les informations des interventions. Une clé Hive absente du JSON est ignorée et ne crée aucune intervention. Sans surcharge, le statut distant est conservé.

Seul le statut est persisté :

- aucun cache complet du JSON ;
- aucune modification du fichier distant ;
- aucun enrichissement local de l'historique.

Un premier chargement nécessite donc le réseau, même si des statuts sont déjà enregistrés.

7.3 Ordre de mise à jour

1. le Domain valide la transition ;
2. le repository demande l'écriture Hive ;
3. l'application attend la réussite de l'écriture ;
4. l'`AsyncNotifier` remplace le statut dans son état ;
5. Riverpod reconstruit la liste et le détail.

En cas d'échec Hive, l'état Riverpod n'est pas modifié. L'ancien statut reste visible et un message informe l'utilisateur. Une box fermée ou une valeur locale inconnue produit explicitement une `LocalStorageFailure`.

8. Gestion des erreurs

Toutes les erreurs exposées au reste de l'application dérivent de `AppFailure`.

Failure	Situation représentée	Présentation utilisateur
<code>NetworkFailure</code>	absence de connexion ou certificat refusé	serveur injoignable
<code>RequestTimeoutFailure</code>	timeout connexion, envoi, réponse ou transformation	serveur trop lent
<code>HttpFailure</code>	réponse HTTP non valide	erreur retournée par le serveur
<code>DataParsingFailure</code>	JSON ou valeur métier incohérente	données reçues invalides
<code>LocalStorageFailure</code>	initialisation, lecture ou écriture Hive impossible	données locales indisponibles

InvalidStatusTransitionFailure	progression après terminée	transition refusée
UnknownFailure	erreur non catégorisée	erreur inattendue

Les `DioException`, exceptions de parsing et erreurs Hive sont interceptées dans la couche Data. Les widgets reçoivent une `Failure` et utilisent une traduction centralisée ; aucune exception technique n'est affichée directement.

Le bouton « Réessayer » exécute `InterventionsNotifier.recharger()`, repasse par loading puis remplace l'erreur par les nouvelles données en cas de succès. Aucun retry automatique, backoff ou pull-to-refresh n'est présent.

9. Tests et validation

9.1 Tests automatisés

La suite finale contient **86 tests réussis** couvrant :

- parsing et mapping du JSON ;
- validation des champs, dates, statuts, priorités et coordonnées ;
- règles Domain et transitions ;
- erreurs Dio : réseau, timeouts, HTTP, réponse inattendue et erreur inconnue ;
- lecture, écriture, écrasement et réouverture Hive ;
- erreurs de lecture/écriture locale ;
- fusion JSON/Hive, y compris une clé locale obsolète ;
- providers Riverpod et ordre de persistance ;
- retry error → loading → data ;
- recherche et combinaisons de filtres ;
- états UI, détail, profil et navigation.

Les tests réseau utilisent un adaptateur Dio contrôlé et ne sollicitent pas le serveur réel. Les tests Hive créent des répertoires temporaires supprimés après chaque scénario. Les tests sont indépendants du stockage utilisateur et de leur ordre d'exécution.

Aucun pourcentage de couverture n'est annoncé, car aucune mesure de couverture n'a été demandée ni calculée.

9.2 Vérifications techniques

La validation du 4 septembre 2026 a produit les résultats suivants :

- `dart format .` : succès ;
- `flutter pub get` : succès ;
- `flutter analyze` : aucune erreur ;
- `flutter test` : 86 tests réussis ;
- `flutter build apk --debug` : APK construit ;
- lancement sur Samsung SM S926U, Android 16/API 36 : installation et démarrage technique réussis.

9.3 Validation manuelle

La recette fonctionnelle manuelle a été réalisée par le développeur sur un Samsung SM S926U sous Android 16 / API 36. Les scénarios suivants ont été validés :

Scénario	Critère attendu
Lancement Android	aucun crash au démarrage
Liste réelle	les 14 interventions sont accessibles sans filtre
Recherche	station, domaine et description filtrent la liste
Filtres	statut et priorité fonctionnent séparément et ensemble
Détail	tous les champs et l'historique sont lisibles
Transition	le statut progresse dans l'ordre autorisé
Retour liste	le statut mis à jour est immédiatement visible
Profil	l'identité correspond au technicien du JSON
Fermeture/relaunch	le statut local est conservé
Affichage mobile	aucun overflow visible

Résultat : la recette tactile et visuelle a été confirmée par le développeur sur l'appareil réel. La liste, la recherche, les filtres, le détail, les transitions de statut, le profil, la persistance après fermeture/relaunch et l'absence d'overflow visible ont été validés.

10. Difficultés et solutions

10.1 Distinguer données de référence et modifications locales

Problème : conserver les changements sans altérer le JSON fourni et sans inventer un cache complet.

Choix : stocker uniquement `interventionId` → statut dans Hive puis fusionner ces surcharges avec chaque nouveau chargement distant.

Conséquence : les statuts survivent au redémarrage, tandis que le JSON reste la référence de la liste et des autres champs.

10.2 Garantir une mise à jour cohérente

Problème : une mise à jour optimiste pouvait afficher un statut non réellement persisté.

Choix : attendre le succès Hive avant de modifier l'état Riverpod.

Conséquence : un échec conserve l'ancien statut et aucune fausse réussite n'est présentée.

10.3 Détecter une lecture Hive indisponible

Problème : `toMap()` sur une box fermée peut renvoyer une map vide, ce qui ressemble à une absence de surcharge.

Choix : contrôler explicitement `Box.isOpen` avant lecture et écriture.

Conséquence : une fermeture anormale devient une `LocalStorageFailure` globale au lieu de masquer des données locales potentiellement attendues.

10.4 Respecter le retry exclusivement manuel

Problème : Riverpod 3 peut relancer automatiquement un provider asynchrone en erreur.

Choix : désactiver explicitement le retry du provider principal.

Conséquence : le seul rechargement est celui déclenché par « Réessayer », conformément au périmètre retenu.

10.5 Afficher deux familles de filtres sur mobile

Problème : les chips de statut et priorité dépassent la largeur d'un téléphone.

Choix : organiser chaque famille dans une ligne à défilement horizontal avec des libellés sémantiques.

Conséquence : tous les filtres restent accessibles sans réduire excessivement leurs zones tactiles.

10.6 Vérifier iOS depuis Windows

Problème : la toolchain Xcode n'est pas disponible sous Windows.

Choix : préserver et contrôler statiquement la cible iOS, puis documenter la vérification restante.

Conséquence : le projet iOS est généré et cohérent, mais sa compilation doit être réalisée sur macOS/Xcode avant remise définitive.

11. Limites

- aucun cache complet ou mode offline-first ;
- réseau obligatoire au premier chargement ;
- aucune synchronisation serveur des statuts ;
- aucune résolution de conflits entre appareils ;
- historique distant non enrichi localement ;
- compilation et exécution iOS non vérifiées sous Windows ;
- aucune fonctionnalité bonus du sujet.

12. Améliorations futures

Les évolutions possibles, hors version livrée, sont :

- cache local complet et consultation hors ligne ;

- synchronisation serveur différée ;
- résolution de conflits ;
- ajout de photos et notes ;
- cartographie des stations ;
- tableau de bord par statut ;
- authentification et gestion de session ;
- interface spécialisée tablette ;
- thème sombre.

13. Conclusion

AgriVista Field couvre le périmètre minimum imposé : consultation du JSON, recherche et filtres, détail, évolution métier du statut, persistance locale, gestion d'état explicite, profil et thème Material 3. La Clean Architecture isole les règles du Domain, le composition root assemble les détails techniques et les tests sécurisent les frontières réseau, parsing, Hive, Riverpod et UI.

Le projet Android est analysé, testé, construit et validé manuellement sur appareil réel. Les éléments restant avant remise sont la compilation de la cible iOS sur macOS/Xcode, l'export du dossier de conception et du présent dossier technique au format PDF, ainsi que l'application de la convention de nommage de l'école.

Annexe A — Matrice de conformité au sujet

Exigence	État	Preuve dans le projet
Dossier de conception	Source présente, export PDF à réaliser	docs/ Dossier_conception_AgriVista_Field.docx
Dépôt Git partagé	Dépôt distant configuré, partage pédagogique à confirmer	remote origin vers DorianArg/AgriVistaEXAM
Flutter Android	Validé techniquement et manuellement	APK debug construit, lancement et recette fonctionnelle validés sur SM S926U
Flutter iOS	Projet iOS généré / compilation à vérifier sur macOS	dossier ios/, projet Xcode et workspace présents
JSON fourni	Conforme	URL centralisée, requête Dio GET en lecture seule
Désérialisation typée	Conforme	DTO Freezed/json_serializable et tests de parsing
Liste	Conforme	InterventionsPage, cartes et test widget

Recherche	Conforme	station, domaine, description et tests de cas limites
Filtre statut	Conforme	quatre critères dont « Tous » et tests
Filtre priorité	Conforme	quatre critères dont « Toutes » et tests
Détail	Conforme	station, domaine, coordonnées, description, date, statut, priorité, historique
Mise à jour du statut	Conforme	use case Domain et transitions testées
Persistance locale	Conforme	box Hive, écriture, réouverture, fusion et tests automatisés
Persistance après redémarrage	Conforme	réouverture Hive testée + fermeture/relaunch Android validée manuellement
Loading / data / error	Conforme	AsyncValue, vues dédiées et tests widgets
Retry manuel	Conforme	retry automatique désactivé et cycle complet testé
Material 3	Conforme	AppTheme, useMaterial3: true
Profil	Conforme	technicien issu du provider principal, test anti-valeur codée en dur
Clean Architecture	Conforme	Data / Domain / Presentation et composition root
Riverpod	Conforme	Provider, AsyncNotifier, Notifier, ref.watch/ref.read
Erreurs typées	Conforme	hiérarchie AppFailure et traductions UI
Dossier technique	Source Markdown créée, export PDF à réaliser	docs/dossier_technique.md

Annexe B — Livraison et versionnement

Le projet a été développé par étapes avec des commits significatifs pour l'initialisation, le Domain et le chargement distant, Hive, la liste, le détail/statut puis le profil et les finitions. Le dépôt distant configuré est :

<https://github.com/DorianArg/AgriVistaEXAM>

Les éléments locaux suivants sont ignorés par Git :

- android/local.properties ;
- .dart_tool/ ;
- build/ et les APK générés ;
- .idea/ ;
- caches et fichiers temporaires Flutter/IDE.

Avant la remise, le développeur doit confirmer le partage du dépôt privé avec l'équipe pédagogique, exporter le dossier de conception et ce dossier technique en PDF, puis nommer les fichiers selon la convention de l'école.